信 息 工 程 学 院

《云计算》实验报告

学院：信息工程学院　**班级**：数科 2102　**姓名**：陈　**学号**:202101　**成绩**:

一、实验题目

Hbase 操作与编程

二、实验目的

通过本实验掌握 HBase 基本操作；通过本实验掌握利用 HBase 提供的 Java API 进行编程对 HBase 进行操作。

三、实验平台

CourseGrading 在线实验环境

四、实验过程及步骤

实验五

启动服务

```
root@master:~# bash /scripts/hbase/start-hbase.sh
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Stopping namenodes on [master]
master: no namenode to stop
slave1: no datanode to stop
slave2: no datanode to stop
slave3: no datanode to stop
Stopping secondary namenodes [0.0.0.0]
0.0.0.0: no secondarynamenode to stop
Starting namenodes on [master]
```

进入 hbase shell

实验报告的内容与格式按任课教师的要求书写。

```
root@master:~# hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/module/hbase-1.1.5/lib/slf4j-log4j12-1.7.
5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/module/hadoop-2.7.1/share/hadoop/common/l
ib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.1.5, r239b80456118175b340b2e562a5568b5c744252e, Sun May  8 20:29:26 PD
T 2016

hbase(main):001:0>
```

创建表

```
hbase(main):001:0> create 'student','name','sex','age','dept','course'
0 row(s) in 1.4210 seconds

=> Hbase::Table - student
```

查看表

```
hbase(main):002:0> list
TABLE
student
1 row(s) in 0.0380 seconds

=> ["student"]
```

查看 student 表属性

```
hbase(main):003:0> describe 'student'
Table student is ENABLED
student
COLUMN FAMILIES DESCRIPTION
{NAME => 'age', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KEE
P_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COM
PRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '655
36', REPLICATION_SCOPE => '0'}
{NAME => 'course', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false',
KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER',
COMPRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '
65536', REPLICATION_SCOPE => '0'}
{NAME => 'dept', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KE
EP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', CO
MPRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65
536', REPLICATION_SCOPE => '0'}
{NAME => 'name', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KE
EP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', CO
MPRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65
536', REPLICATION_SCOPE => '0'}
{NAME => 'sex', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KEE
P_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COM
PRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '655
36', REPLICATION_SCOPE => '0'}
5 row(s) in 0.2150 seconds

hbase(main):004:0>
```

插入数据

```
hbase(main):005:0* put 'student','1606001','name','WangWei'
0 row(s) in 0.1250 seconds

hbase(main):006:0> put 'student','1606001','sex','male'
0 row(s) in 0.0180 seconds

hbase(main):007:0> put 'student','1606001','age','20'
0 row(s) in 0.0150 seconds

hbase(main):008:0> put 'student','1606001','dept','CS'
0 row(s) in 0.0090 seconds

hbase(main):009:0> put 'student','1606001','course:math','80'
0 row(s) in 0.0110 seconds
```

查看数据

```
hbase(main):010:0> get 'student','1606001'
COLUMN                  CELL
 age:                   timestamp=1699016418240, value=20
 course:math            timestamp=1699016437989, value=80
 dept:                  timestamp=1699016426144, value=CS
 name:                  timestamp=1699016388420, value=WangWei
 sex:                   timestamp=1699016410270, value=male
5 row(s) in 0.0550 seconds

hbase(main):011:0> get 'student','1606001','sex'
COLUMN                  CELL
 sex:                   timestamp=1699016410270, value=male
1 row(s) in 0.0130 seconds
```

插入数据

```
hbase(main):012:0> put 'student','1606002','name','ZhangLi'
0 row(s) in 0.0150 seconds

hbase(main):013:0> put 'student','1606002','sex','female'
0 row(s) in 0.0150 seconds

hbase(main):014:0> put 'student','1606002','age','19'
0 row(s) in 0.0090 seconds

hbase(main):015:0> put 'student','1606002','dept','EE'
0 row(s) in 0.0110 seconds

hbase(main):016:0> put 'student','1606002','course:math','85'
0 row(s) in 0.0090 seconds
```

查看数据库中所有数据

```
hbase(main):017:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=age:, timestamp=1699016418240, value=20
 1606001               column=course:math, timestamp=1699016437989, value=80
 1606001               column=dept:, timestamp=1699016426144, value=CS
 1606001               column=name:, timestamp=1699016388420, value=WangWei
 1606001               column=sex:, timestamp=1699016410270, value=male
 1606002               column=age:, timestamp=1699016573176, value=19
 1606002               column=course:math, timestamp=1699016589976, value=85
 1606002               column=dept:, timestamp=1699016580436, value=EE
 1606002               column=name:, timestamp=1699016540133, value=ZhangLi
 1606002               column=sex:, timestamp=1699016560003, value=female
2 row(s) in 0.0420 seconds
```

删除数据并检验

```
hbase(main):018:0> delete 'student','1606001','age'
0 row(s) in 0.0450 seconds

hbase(main):019:0> deleteall 'student','1606002'
0 row(s) in 0.0130 seconds

hbase(main):020:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=course:math, timestamp=1699016437989, value=80
 1606001               column=dept:, timestamp=1699016426144, value=CS
 1606001               column=name:, timestamp=1699016388420, value=WangWei
 1606001               column=sex:, timestamp=1699016410270, value=male
1 row(s) in 0.0420 seconds

hbase(main):021:0>
```

删除表并检验

```
hbase(main):021:0> disable 'student'
0 row(s) in 2.2930 seconds

hbase(main):022:0> drop 'student'
0 row(s) in 1.4370 seconds

hbase(main):023:0> list
TABLE
0 row(s) in 0.0090 seconds

=> []
```

数据准备

```
hbase(main):024:0> create 'student',{ NAME=>'name', VERSIONS=>5},{ NAME=>'score', VERSIONS=>3}
0 row(s) in 1.2290 seconds

=> Hbase::Table - student
hbase(main):025:0> put 'student','1606001','name','Jack'
0 row(s) in 0.0180 seconds

hbase(main):026:0> put 'student','1606001','name','Jack1'
0 row(s) in 0.0080 seconds

hbase(main):027:0> put 'student','1606001','name','Jack2'
0 row(s) in 0.0060 seconds

hbase(main):028:0> put 'student','1606001','name','Jack3'
0 row(s) in 0.0060 seconds

hbase(main):029:0> put 'student','1606001','name','Jack4'
0 row(s) in 0.0060 seconds

hbase(main):030:0> put 'student','1606001','name','Jack5'
0 row(s) in 0.0090 seconds

hbase(main):031:0> put 'student','1606001','name','Jack6'
0 row(s) in 0.0060 seconds

hbase(main):032:0> put 'student','1606001','score','80'
0 row(s) in 0.0050 seconds

hbase(main):033:0> put 'student','1606001','score','81'
0 row(s) in 0.0060 seconds

hbase(main):034:0> put 'student','1606001','score','82'
0 row(s) in 0.0070 seconds
```

查询行键为 1606001 学生姓名字段的最近 5 个版本

```
hbase(main):037:0> get 'student','1606001',{COLUMN=>'name',VERSIONS=>5}
COLUMN                    CELL
 name:                    timestamp=1699016785571, value=Jack6
 name:                    timestamp=1699016785546, value=Jack5
 name:                    timestamp=1699016785524, value=Jack4
 name:                    timestamp=1699016785502, value=Jack3
 name:                    timestamp=1699016785479, value=Jack2
5 row(s) in 0.0150 seconds
```

查询行键为 1606001 学生姓名字段的最近 3 个版本

```
hbase(main):038:0> get 'student','1606001',{COLUMN=>'name',VERSIONS=>3}
COLUMN                    CELL
 name:                    timestamp=1699016785571, value=Jack6
 name:                    timestamp=1699016785546, value=Jack5
 name:                    timestamp=1699016785524, value=Jack4
3 row(s) in 0.0100 seconds
```

查询行键为 1606001 学生得分字段的最近 3 个版本

```
hbase(main):039:0> get 'student','1606001',{COLUMN=>'score',VERSIONS=>3}
COLUMN                    CELL
 score:                   timestamp=1699016787815, value=84
 score:                   timestamp=1699016785652, value=83
 score:                   timestamp=1699016785631, value=82
3 row(s) in 0.0080 seconds
```

查询行键为 1606001 学生得分字段的最近 2 个版本

```
hbase(main):040:0> get 'student','1606001',{COLUMN=>'score',VERSIONS=>2}
COLUMN                    CELL
 score:                   timestamp=1699016787815, value=84
 score:                   timestamp=1699016785652, value=83
2 row(s) in 0.0110 seconds
```

作业 1

1. 在 HBase 中，创建 mytale，包含两个列 ca,cb

```
hbase(main):044:0> create 'mytable','ca','cb'
0 row(s) in 1.2590 seconds
```

```
hbase(main):046:0> describe 'mytable'
Table mytable is ENABLED
mytable
COLUMN FAMILIES DESCRIPTION
{NAME => 'ca', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KEEP_DELETED_CELLS
 => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COMPRESSION => 'NONE', MIN_VERSI
ONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_SCOPE => '0'}
{NAME => 'cb', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KEEP_DELETED_CELLS
 => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COMPRESSION => 'NONE', MIN_VERSI
ONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_SCOPE => '0'}
2 row(s) in 0.0310 seconds
```

2. 在 HBase 中，依次设置表 mytable 设为无效和有效

```
hbase(main):048:0> disable 'mytable'
0 row(s) in 2.2720 seconds

hbase(main):049:0> enable 'mytable'
0 row(s) in 1.3910 seconds
```

3. HBase 中，往表 mytable 插入两行数据， 第一条数据：行键为 110，列为 ca:name，值为 rose，第二条数据：key 为 111，列为 cb:age，值为 20。

```
hbase(main):050:0> put 'mytable','110','ca:name','rose'
0 row(s) in 0.0260 seconds

hbase(main):051:0> put 'mytable','111','cb:age','20'
0 row(s) in 0.0130 seconds

hbase(main):052:0> scan 'mytable'
ROW                       COLUMN+CELL
 110                      column=ca:name, timestamp=1699017421891, value=rose
 111                      column=cb:age, timestamp=1699017434443, value=20
2 row(s) in 0.0180 seconds
```

4. 在 HBase 中，查看表 mytable 中 key 为’111’的所有数据

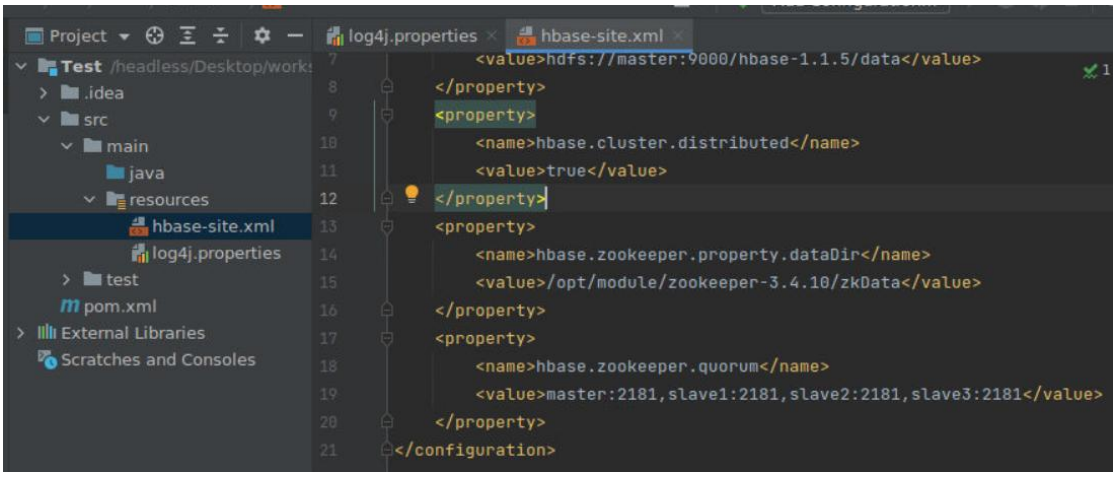
```
hbase(main):053:0> get 'mytable','111'
COLUMN                                  CELL
 cb:age                                 timestamp=1699017434443, value=20
1 row(s) in 0.0130 seconds
```

实验六

启动服务

```
root@master:~# bash /scripts/hbase/start-hbase.sh
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... already running as process 81.
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... already running as process 64.
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... already running as process 64.
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.4.10/bin/../conf/zoo.cfg
Starting zookeeper ... already running as process 64.
Stopping namenodes on [master]
master: stopping namenode
slave2: stopping datanode
slave1: stopping datanode
slave3: stopping datanode
Stopping secondary namenodes [0.0.0.0]
0.0.0.0: stopping secondarynamenode
Starting namenodes on [master]
master: starting namenode, logging to /opt/module/hadoop-2.7.1/logs/hadoop-root-
namenode-master.out
slave3: starting datanode, logging to /opt/module/hadoop-2.7.1/logs/hadoop-root-
datanode-slave3.out
slave2: starting datanode, logging to /opt/module/hadoop-2.7.1/logs/hadoop-root-
datanode-slave2.out
slave1: starting datanode, logging to /opt/module/hadoop-2.7.1/logs/hadoop-root-
```

导入依赖



连接 Hbase 数据库

```
DEBUG - DataTransferProtocol not using SaslPropertiesResolver, no QOP found in configuration for dfs.data
DEBUG - Codec=org.apache.hadoop.hbase.codec.KeyValueCodec@895e367, compressor=null, tcpKeepAlive=true, tc
Connect to HBase successfully!
DEBUG - Reading reply sessionid:0x28b98a1c1480006, packet:: clientPath:null serverPath:null finished:fals
DEBUG - Reading reply sessionid:0x28b98a1c1480006, packet:: clientPath:null serverPath:null finished:fals
DEBUG - Use SIMPLE authentication for service ClientService, sasl=false
DEBUG - Connecting to master/100.64.0.3:16020
```

查看表是否存在

```
DEBUG - Reading reply sessionid:0x38b98a1c1510008, packet:: clientPath:null se
DEBUG - Use SIMPLE authentication for service ClientService, sasl=false
DEBUG - Connecting to master/100.64.0.3:16020
table student does exist!
 INFO - Closing zookeeper sessionid=0x38b98a1c1510008
DEBUG - Closing session: 0x38b98a1c1510008
```

```
DEBUG - Reading reply sessionid:0x28b98a1c1480006, packet:: clientPath:null serverPath
DEBUG - Use SIMPLE authentication for service ClientService, sasl=false
DEBUG - Connecting to master/100.64.0.3:16020
table teacher don't exist!
 INFO - Closing zookeeper sessionid=0x28b98a1c1480006
DEBUG - Closing session: 0x28b98a1c1480006
DEBUG - Closing client for session: 0x28b98a1c1480006
```

```
hbase(main):011:0> list
TABLE
student
1 row(s) in 0.0100 seconds

=> ["student"]
```

创建表

```
DEBUG - Connecting to master/100.64.0.3:16000
 INFO - Created test
table is created successfully! test
 INFO - Closing master protocol: MasterService
 INFO - Closing zookeeper sessionid=0x28b98a1c1480007
```

```
hbase(main):012:0> list
TABLE
student
test
2 row(s) in 0.0120 seconds
```

查看表

```
DEBUG - Connecting to master/100.64.0.3:16000
student
test
 INFO - Closing master protocol: MasterService
```

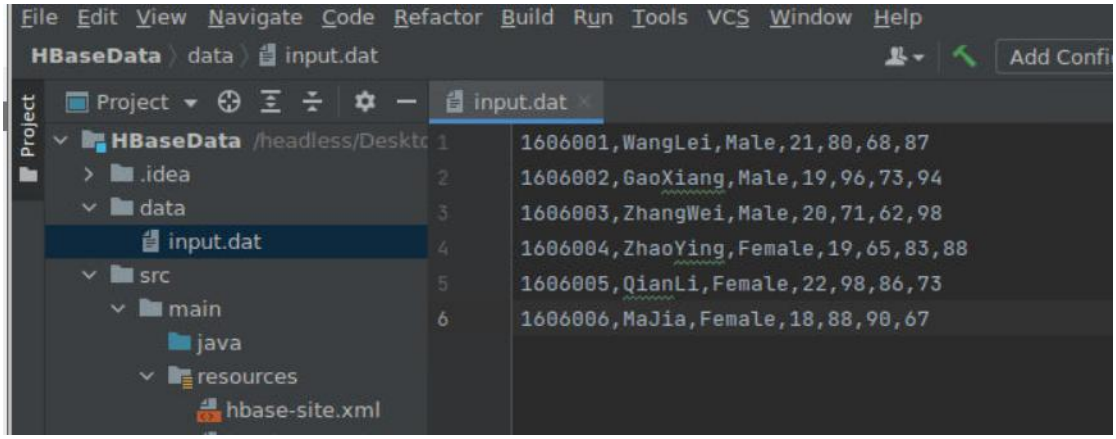
删除表

```
 INFO - Disabled test
 INFO - Deleted test
table test is deleted successfully!
```

```
=> ["student", "test"]
hbase(main):013:0> list
TABLE
student
1 row(s) in 0.0080 seconds
```

准备数据

```
hbase(main):002:0> list
TABLE
student
1 row(s) in 0.0270 seconds
```



```
DEBUG - Connecting to master/100.64.0.3:16020
processing line:1606002,GaoXiang,Male,19,96,73,94
processing line:1606003,ZhangWei,Male,20,71,62,98
processing line:1606004,ZhaoYing,Female,19,65,83,88
processing line:1606005,QianLi,Female,22,98,86,73
processing line:1606006,MaJia,Female,18,88,90,67
INFO - Closing zookeeper sessionid=0x28b98a1c148000a
```

```
hbase(main):003:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=Age:, timestamp=1699074327794, value=21
 1606001               column=Course:Chinese, timestamp=1699074327794, value=87
 1606001               column=Course:English, timestamp=1699074327794, value=68
 1606001               column=Course:Math, timestamp=1699074327794, value=80
 1606001               column=Name:, timestamp=1699074327794, value=WangLei
 1606001               column=Sex:, timestamp=1699074327794, value=Male
 1606002               column=Age:, timestamp=1699074327832, value=19
 1606002               column=Course:Chinese, timestamp=1699074327832, value=94
 1606002               column=Course:English, timestamp=1699074327832, value=73
 1606002               column=Course:Math, timestamp=1699074327832, value=96
 1606002               column=Name:, timestamp=1699074327832, value=GaoXiang
 1606002               column=Sex:, timestamp=1699074327832, value=Male
 1606003               column=Age:, timestamp=1699074327839, value=20
 1606003               column=Course:Chinese, timestamp=1699074327839, value=98
 1606003               column=Course:English, timestamp=1699074327839, value=62
 1606003               column=Course:Math, timestamp=1699074327839, value=71
 1606003               column=Name:, timestamp=1699074327839, value=ZhangWei
```

查询数据

```
1606006 Sex     Female
Scan part data1...
1606002 Age     19
1606002 Course  Chinese 94
1606002 Course  English 73
1606002 Course  Math    96
1606002 Name        GaoXiang
1606002 Sex     Male
1606005 Age     22
1606005 Course  Chinese 73
1606005 Course  English 86
1606005 Course  Math    98
1606005 Name        QianLi
1606005 Sex     Female
Scan part data2...
1606005 Age     22
1606005 Course  Chinese 73
1606005 Course  English 86
1606005 Course  Math    98
1606005 Name        QianLi
1606005 Sex     Female
1606006 Age     18
1606006 Course  Chinese 67
1606006 Course  English 90
1606006 Course  Math    88
1606006 Name        MaJia
```

修改数据

```
Connect to HBase successfully!
processing line:1606001,WangLei,Male,20,83,91,76
DEBUG - Reading reply sessionid:0x18b98a1c1480003, packet:: cli
DEBUG - Reading reply sessionid:0x18b98a1c1480003, packet:: cli
DEBUG - Use SIMPLE authentication for service ClientService, sa
DEBUG - Connecting to master/100.64.0.3:16020
processing line:1606002,GaoXiang,Male,20,98,89,91
processing line:1606003,ZhangWei,Male,19,99,83,74
processing line:1606004,ZhaoYing,Female,18,91,73,76
processing line:1606005,QianLi,Female,17,94,89,88
processing line:1606006,MaJia,Female,21,81,83,83
INFO - Closing zookeeper sessionid=0x18b98a1c1480003
```

```
hbase(main):004:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=Age:, timestamp=1699074561528, value=20
 1606001               column=Course:Chinese, timestamp=1699074561528, value=76
 1606001               column=Course:English, timestamp=1699074561528, value=91
 1606001               column=Course:Math, timestamp=1699074561528, value=83
 1606001               column=Name:, timestamp=1699074561528, value=WangLei
 1606001               column=Sex:, timestamp=1699074561528, value=Male
 1606002               column=Age:, timestamp=1699074561554, value=20
 1606002               column=Course:Chinese, timestamp=1699074561554, value=91
 1606002               column=Course:English, timestamp=1699074561554, value=89
 1606002               column=Course:Math, timestamp=1699074561554, value=98
 1606002               column=Name:, timestamp=1699074561554, value=GaoXiang
 1606002               column=Sex:, timestamp=1699074561554, value=Male
 1606003               column=Age:, timestamp=1699074561559, value=19
 1606003               column=Course:Chinese, timestamp=1699074561559, value=74
 1606003               column=Course:English, timestamp=1699074561559, value=83
 1606003               column=Course:Math, timestamp=1699074561559, value=99
 1606003               column=Name:, timestamp=1699074561559, value=ZhangWei
 1606003               column=Sex:, timestamp=1699074561559, value=Male
```

删除数据

```
1
DEBUG - Reading reply sessionid:0x38b98a1c151000e, packet:: clientPath:null serverPath:null finished:fal
DEBUG - Reading reply sessionid:0x38b98a1c151000e, packet:: clientPath:null serverPath:null finished:fal
DEBUG - Use SIMPLE authentication for service ClientService, sasl=false
DEBUG - Connecting to master/100.64.0.3:16020
A cell is deleted!
Input 0/1 for:
```

```
hbase(main):007:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=Age:, timestamp=1699074561528, value=20
 1606001               column=Course:Chinese, timestamp=1699074561528, value=76
 1606001               column=Course:English, timestamp=1699074561528, value=91
 1606001               column=Name:, timestamp=1699074561528, value=WangLei
 1606001               column=Sex:, timestamp=1699074561528, value=Male
 1606002               column=Age:, timestamp=1699074561554, value=20
```

```
Connect to HBase successfully!
Input 0^4 for:
    0. Exit the program
    1. Delete a cell.
    2. Delete a cluster.
    3. Delete a row.
2
DEBUG - Reading reply sessionid:0x28b98a1c148000c, packet:: cli
DEBUG - Reading reply sessionid:0x28b98a1c148000c, packet:: cli
DEBUG - Use SIMPLE authentication for service ClientService, sa
DEBUG - Connecting to master/100.64.0.3:16020
A cluster is deleted!
Input 0^4 for:
    0. Exit the program
    1. Delete a cell.
    2. Delete a cluster.
    3. Delete a row.
0
INFO - Closing zookeeper sessionid=0x28b98a1c148000c
```

```
hbase(main):008:0> scan 'student'
ROW                    COLUMN+CELL
 1606001               column=Age:, timestamp=1699074561528, value=20
 1606001               column=Name:, timestamp=1699074561528, value=Wangl
 1606001               column=Sex:, timestamp=1699074561528, value=Male
 1606002               column=Age:, timestamp=1699074561554, value=20
 1606002               column=Course:Chinese, timestamp=1699074561554, va
```

```
Connect to HBase successfully!
Input 0^4 for:
    0. Exit the program
    1. Delete a cell.
    2. Delete a cluster.
    3. Delete a row.
3
DEBUG - Reading reply sessionid:0x18b98a1c1
DEBUG - Reading reply sessionid:0x18b98a1c1
DEBUG - Use SIMPLE authentication for servi
DEBUG - Connecting to master/100.64.0.3:160
A row is deleted!
Input 0^4 for:
    0. Exit the program
    1. Delete a cell.
    2. Delete a cluster.
    3. Delete a row.
0
 INFO - Closing zookeeper sessionid=0x18b98
DEBUG - Closing session: 0x18b98a1c1480004
DEBUG - Closing client for session: 0x18b98
```

```
hbase(main):009:0> scan 'student'
ROW                    COLUMN+CELL
 1606002               column=Age:, timestamp=1699074561554, value=20
 1606002               column=Course:Chinese, timestamp=1699074561554, value=9
 1606002               column=Course:English, timestamp=1699074561554, value=8
 1606002               column=Course:Math, timestamp=1699074561554, value=98
 1606002               column=Name:, timestamp=1699074561554, value=GaoXiang
 1606002               column=Sex:, timestamp=1699074561554, value=Male
 1606003               column=Age:, timestamp=1699074561559, value=19
 1606003               column=Course:Chinese, timestamp=1699074561559, value=7
```

作业

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.hbase.*;
import org.apache.hadoop.hbase.client.*;
import org.apache.hadoop.hbase.util.Bytes;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
```

```
import java.util.ArrayList;
import java.util.Scanner;

public class StudentDB {
	public static Configuration configuration;
	public static Connection connection;
	public static Admin admin;
	public static Scanner sc;

	//main
	public static void main(String[] args) {
		init();
		sc = new Scanner(System.in);
		try {
			for (;;) {

System.out.println("----------------------------------------------------");
				System.out.println("Welcome to the student information management system!");
				System.out.println("Input 0~3 for:");
				System.out.println("\t0.Insert data");
				System.out.println("\t1.Review data");
				System.out.println("\t2.Delete data");
				System.out.println("\t3.Exit the system");

System.out.println("----------------------------------------------------");
				System.out.print("Input your option:");
				int choice = sc.nextInt();
				switch (choice) {
					case 0:
						insertData();
						break;
					case 1:
						reviewData();
						break;
					case 2:
						deleteData();
						break;
					case 3:
						System.out.println("Thank you for your using!");
						break;
					default:
						System.out.println("Error input!");
```

```
                    break;
                }
                if (choice == 3) break;
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            close();
        }
    }

    public static void init(){
        configuration    = HBaseConfiguration.create();
        try{
            connection                                                    =
ConnectionFactory.createConnection(configuration);
            admin = connection.getAdmin();
        }catch (IOException e){
            e.printStackTrace();
        }
        System.out.println("Connect to HBase successfully!");
    }

    //insert
    public static void insertData() throws IOException{
        String str;
        System.out.print("Rowkey:");
        str = sc.next();
        Put put = new Put(str.getBytes());

        System.out.print("No:");
        str = sc.next();
        put.addColumn("No".getBytes(),"".getBytes(),str.getBytes());

        System.out.print("Name:");
        str = sc.next();
        put.addColumn("Name".getBytes(),"".getBytes(),str.getBytes());

        System.out.print("Sex:");
        str = sc.next();
        put.addColumn("Sex".getBytes(),"".getBytes(),str.getBytes());

        System.out.print("Age:");
        str = sc.next();
```

```
            put.addColumn("Age".getBytes(),"".getBytes(),str.getBytes());

            System.out.print("Dept:");
            str = sc.next();
            put.addColumn("Dept".getBytes(),"".getBytes(),str.getBytes());

            System.out.print("Score of Math:");
            str = sc.next();

put.addColumn("Course".getBytes(),"Math".getBytes(),str.getBytes());

            System.out.print("Score of DS:");
            str = sc.next();

put.addColumn("Course".getBytes(),"DS".getBytes(),str.getBytes());

            Table                                   table                                   =
connection.getTable(TableName.valueOf("student"));

            table.put(put);
            System.out.println("Insert successfully!");
        }

        //review
        public static void reviewData(){
            System.out.println("----------------------------------------");
            System.out.println("Input 0~3 for:");
            System.out.println("\t0.Review latest data of one cell");
            System.out.println("\t1.Review all versions data of one cell");
            System.out.println("\t2.Review one row");
            System.out.println("\t3.Review whole table");
            System.out.println("----------------------------------------");
            System.out.print("Input your option:");
            int choice = sc.nextInt();
            switch (choice) {
                case 0:
                    latestData();
                    break;
                case 1:
                    allVersionData();
                    break;
                case 2:
                    rowData();
                    break;
```

```
                case 3:
                        wholeTable();
                        break;
                default:
                        System.out.println("Error input!");
                        break;
        }
    }

    public static void latestData() throws IOException {
        Table t = connection.getTable(TableName.valueOf("student"));
        System.out.print("Input rowkey:");
        String rowKey = sc.next();
        System.out.print("Input family:");
        String family = sc.next();
        System.out.print("Input qualifier:");
        String qualifier = sc.next();
        if (qualifier.equals("\n")) qualifier = "";
        Get get = new Get(Bytes.toBytes(rowKey));
        //get.setTimeRange(startTime,endTime);
        Result res = t.get(get);
        for(Cell cell : res.rawCells()) {
                System.out.print(Bytes.toString(CellUtil.cloneRow(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneFamily(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneQualifier(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneValue(cell))+"\n");
        }
    }
    public static void allVersionData() throws IOException {

    }
    public static void rowData() throws IOException {
        Table t = connection.getTable(TableName.valueOf("student"));
        System.out.print("Input rowkey:");
        String rowKey = sc.next();
        Get get = new Get(Bytes.toBytes(rowKey));
        Result res = t.get(get);
        for(Cell cell : res.rawCells()) {
                System.out.print(Bytes.toString(CellUtil.cloneRow(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneFamily(cell)));
```

```
System.out.print("\t"+Bytes.toString(CellUtil.cloneQualifier(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneValue(cell))+"\n");
            }
      }       public static void wholeTable(){
            try {
                  Table                                    t                                    =
connection.getTable(TableName.valueOf("student"));
                  ResultScanner rs = t.getScanner(new Scan());
                  for(Result r: rs) {
                        for(Cell cell: r.rawCells()) {

System.out.print(Bytes.toString(CellUtil.cloneRow(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneFamily(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneQualifier(cell)));

System.out.print("\t"+Bytes.toString(CellUtil.cloneValue(cell))+"\n");
                        }
                  }
            } catch (IOException e) {
                  e.printStackTrace();
            }
      }

      //delete
      public static void deleteData(){
            System.out.println("-------------------");
            System.out.println("Input 0~2 for:");
            System.out.println("\t0.Delete one cell");
            System.out.println("\t1.Delete one row");
            System.out.println("\t2.Delete all data");
            System.out.println("-------------------");
            System.out.print("Input your option:");
            int choice = sc.nextInt();
            switch (choice) {
                  case 0:
                        delCell();
                        break;
                  case 1:
                        delRow();
                        break;
```

```
                case 2:
                        delAllData();
                        break;
                default:
                        System.out.println("Error input!");
                        break;
        }
    }

    public static void delCell(){
        try {
                String rowKey, family, qualifier;
                System.out.print("Delete rowkey:");
                rowKey = sc.next();
                System.out.print("Delete family:");
                family = sc.next();
                System.out.print("Delete qualifier:");
                qualifier = sc.next();
                if(qualifier.equals("\n")) qualifier = "";

                Table                                   t                                   =
connection.getTable(TableName.valueOf("student"));
                Delete delete = new Delete(Bytes.toBytes(rowKey));
                delete.addColumn(Bytes.toBytes(family),
Bytes.toBytes(qualifier));
                t.delete(delete);
                System.out.println("Delete cell successfully!");
        } catch (IOException e) {
                e.printStackTrace();
        }
    }
    public static void delRow(){
        try {
                String rowKey;
                System.out.print("Delete rowkey:");
                rowKey = sc.next();

                Table                                   t                                   =
connection.getTable(TableName.valueOf("student"));
                Delete delete = new Delete(Bytes.toBytes(rowKey));
                t.delete(delete);
                System.out.println("Delete row successfully!");
        } catch (IOException e) {
                e.printStackTrace();
```

```
        }
    }
    public static void delAllData() {
        try {
            ArrayList<String> rowKeys = new ArrayList<>();
            Table t =
connection.getTable(TableName.valueOf("student"));
            ResultScanner rs = t.getScanner(new Scan());
            for (Result r : rs) {
                rowKeys.add(Bytes.toString(r.getRow()));
            }

            for (String rowKey : rowKeys){
                Delete delete = new Delete(Bytes.toBytes(rowKey));
                t.delete(delete);
            }
            System.out.println("Delete all data successfully!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
        public static void close(){
        try{
            if(admin != null){
                admin.close();
            }
            if(connection != null){
                connection.close();
            }
        }catch (IOException e){
            e.printStackTrace();
        }
    }
}
```

五、实验总结

通过本次实验，我学习到了许多关于 Hbase 的操作，学习了有关于 Hbase 的一些接口，并进行验证性实验，知道了如何使用 HBase 提供的 API 连接 HBase 数据库，进行数据的增删改查操作。并自己编写了关于使用 API 进行数据操作的项目，不过在关于查找最新数据和所有版本数据上遇到问题，暂未解决。